[cmp653 > final > defense]

Privacy-Budget Consumption in Repeated Aggregate SQL Workloads

> a statistical model _

refik can öztaş // N25142279

hacettepe university | ankara

> 01 ::: aggregate queries are not safe

// intuition (wrong)

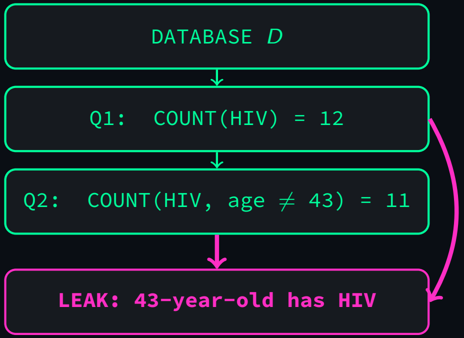
“aggregates return totals, not rows – safe.”

// reality

two overlapping COUNTs isolate one person.

```
– 12 patients with HIV
SELECT COUNT(*) FROM patients
WHERE zip='06897' AND
      disease='HIV';
– 11 with HIV, age <> 43
SELECT COUNT(*) FROM patients
WHERE zip='06897' AND
      disease='HIV' AND
      age <> 43;
```

$\Rightarrow$ the 43-year-old in 06897 has HIV.



Dinur & Nissim 2003:

$O(n)$ noisy aggregates reconstruct the whole DB.

> 02 ::: differential privacy in 30 seconds

ε-differential privacy

$\mathcal{M}$ is ε-DP if for any $D \sim D'$ (differ by 1 row):

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(D') \in S]$$

// Laplace mechanism

$$\tilde{f}(D) = f(D) + \eta, \quad \eta \sim \mathrm{Lap}(\Delta f / \varepsilon)$$

// budget composes additively across queries.

// example: Q = COUNT(*) = 1000

ε	scale	answer
1.0	± 1	999–1001
0.1	± 10	990–1010
0.01	± 100	900–1100

smaller $\varepsilon \Rightarrow$ stronger privacy + more noise

the engineering question

A finite total budget $\varepsilon_{\text{total}}$ is consumed as the workload runs.
HOW does it degrade? Predicting that before deployment is what we do.

> 03 ::: milestone version (what i had)

- Python middleware: SQL parser + sensitivity analyzer + Laplace mechanism
- Budget ledger with exact-repeat caching
- TPC-H benchmark, 4 workload families (W1–W4)
- 28 unit tests, working prototype

milestone headline

"Workload-aware caching saves 100× budget on a repetitive workload by exploiting DP's post-processing property."

// i thought this was the contribution.

> 04 ::: based on the review i received

// points the reviewer raised:

- caching's role in DP wasn't clear
- Algorithm 1 was too thin
- $1 - 1/k$ left unexplained
- AVG mechanism wasn't justified
- caching as a contribution is debatable

// and a structural ask:

"exact-repeat assumptions are strong.
budget and temporality should be considered together.
leakage / savings models are limited.

MECHANISM
(caching)

= debatable

↓

MODEL
(workload structure → budget)

= contribution

// the framing of the contribution had to flip.

> 05 ::: research question, reframed

old: “does caching help?” // trivial

new:

Given a workload distribution and a privacy budget, can a closed-form statistical model predict the realized budget consumption and per-query utility before any query is issued?

Caching is now the corner case of the model, not the contribution.

// five concrete deliverables:

1. Analytical model (workload structure → budget)
2. Temporal extension (staleness + updates)
3. Predictive allocator (new mechanism)
4. Leakage analysis (MIA + reconstruction)
5. Empirical campaign (model validation)

> 06 ::: the model in one slide

// setup: k queries drawn i.i.d. from $\{p_i\}_{i=1}^{m}$.

prop 1 – expected unique queries

$$\mathbb{E}[u_k] = \sum_{i=1}^{m} \left[1 - (1 - p_i)^k\right]$$

prop 2 – workload-aware budget

$$\mathbb{E}[\varepsilon_{\mathrm{wa}}] = \varepsilon_q \cdot \mathbb{E}[u_k]$$

two limits:

- $\alpha \to \infty$: $S(k) = 1 - 1/k$
  milestone's toy formula
- $\alpha \to 0$: $S(k) \to 0$

// concrete example: dashboard with 5 tiles

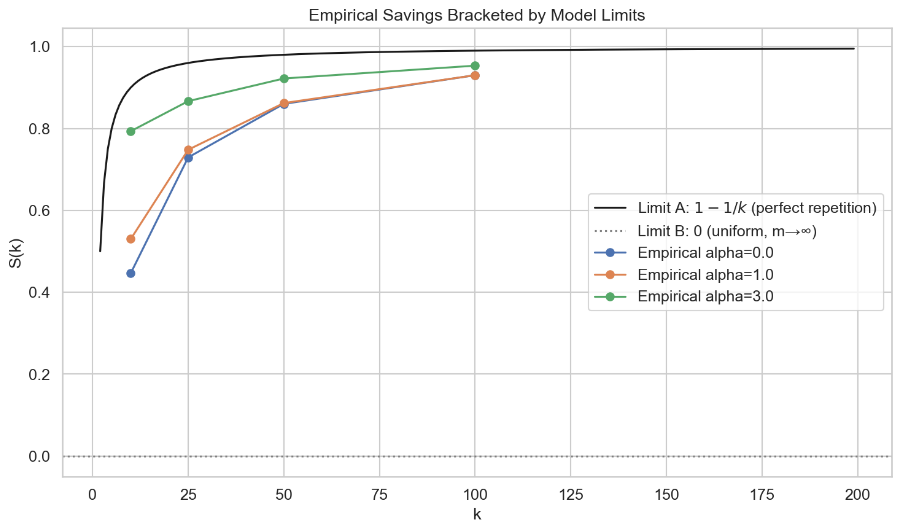
tile	prob p_i	meaning
A	0.42	"urgent"
B	0.26	"high"
C	0.17	"medium"
D	0.10	"low"
E	0.05	"other"

Zipf($\alpha = 1$) over $m = 5$

run $k = 100$ queries:
prop 1 predicts $\mathbb{E}[u_k] \approx 4.97$
prop 2 predicts budget $= \varepsilon_q \cdot 4.97$

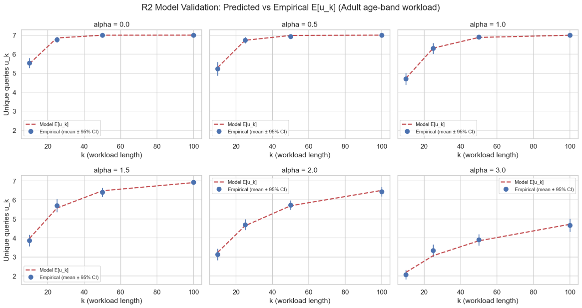
$\Rightarrow$ 95% savings vs naive's $100\varepsilon_q$

> 07 ::: the two limits, empirically



dashed: model prediction | markers: empirical mean over 30 trials

> 08 ::: main grid – 21 of 24 cells within 3%



// markers should land on the dashed identity

// setup

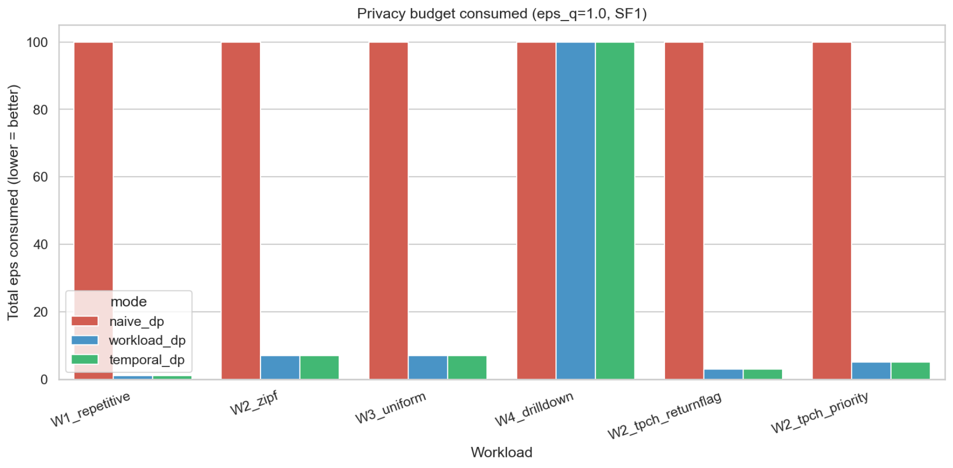
- ▶ $\alpha \in \{0, 0.5, 1, 1.5, 2, 3\}$
- ▶ $k \in \{10, 25, 50, 100\}$
- ▶ 30 trials per cell
- ▶ $\rightarrow$ 720 trials

// headline

- ▶ 21 / 24 cells within 3%
- ▶ Worst case 8.6% at small k, high α

Predict privacy cost from the workload distribution – before any query runs.

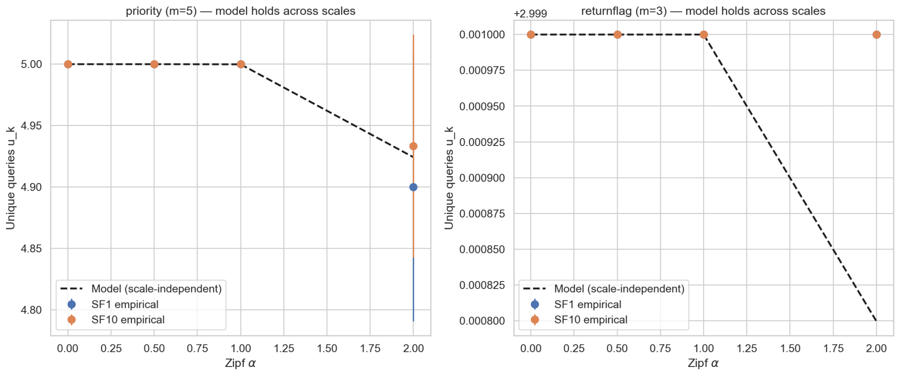
> 09 ::: full benchmark — 6 workloads x 3 modes



$\varepsilon_q = 1$, total budget $B = 100$, $k = 100$, 30 trials per cell.

red = naive (consumes full B) | blue = workload-DP | green = temporal-DP

> 10 ::: scale-independent (sf=1 vs sf=10)



TPC-H SF=1 (6M rows) vs SF=10 (60M rows)

Empirical $\mathbb{E}[u_k]$ identical within 0.03 units

$\Rightarrow$ the model captures workload structure, not data scale.

> 11 ::: heterogeneous workload – exact match

// W1 + W4 hybrid: fraction f repetitive, $1-f$ unique.
by construction, true $u_k = 1 + (1-f) \cdot k$.

f	true u_k	empirical ε	predicted $\varepsilon_q \cdot u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

$\Rightarrow$ empirical matches prediction to the cent in every cell.

// closed form $\mathbb{E}[\varepsilon_{\mathrm{wa}}] = \varepsilon_q \cdot u_k$ holds without requiring Zipf parametricity.

> 12 ::: predictive allocator – the new mechanism

// use the model's $\mathbb{E}[u_k]$ estimate at runtime to choose ε_q:

1. observe template frequencies online (first 3–10 queries: warmup)
2. estimate $\hat{U} = \mathbb{E}[u_k \mid \hat{p}]$ from Proposition 1
3. allocate $\varepsilon_q^{\text{pred}} = B/\hat{U}$ (matches the offline optimum)

// example: $B = 10$, $k = 100$. Naive picks $\varepsilon_q = 10/100 = 0.1$. If we observe $\hat{U} = 7$ unique templates, predictive picks $\varepsilon_q = 10/7 \approx 1.43$ – 14× tighter noise per release.

novel because

PINQ / PrivateSQL / Chorus / DOP-SQL all fix ε_q offline.
This is the first DP-SQL mechanism that adapts ε_q at runtime from a learned model.

B	Mode	MAE	vs naive
1	naive	101.1	–
	predictive	68.7	−32%
10	naive	10.1	–
	predictive	8.3	−18%

> 13a ::: semantic cache – the idea

// step 1: SQL → AST

```
SELECT COUNT(*)
FROM patients
WHERE age > 30
```

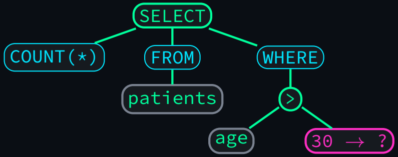
// step 2: normalize literals → ?

```
SELECT COUNT(*)
FROM patients
WHERE age > ?
```

// step 3: compare two ASTs

- Tree Kernel: count shared subtrees
- AST embed: cosine on 384-dim vec

// step 1 visualized (AST tree):



// literal '30' becomes a placeholder so two queries with different ages share the same AST.

> 13b ::: semantic cache – honest negative

false positive (admits wrong match)

```
Q1:  SELECT COUNT(*) FROM adult WHERE
     age ≥ 20 AND age < 30
Q2:  SELECT COUNT(*) FROM adult WHERE
     age ≥ 40 AND age < 50
```

Same AST template after literal removal.
Very different true counts!
⇒ cache returns wrong answer by 1000s

false negative (misses true match)

```
Q1:  ...  WHERE a AND b
Q2:  ...  WHERE b AND a
```

Tree Kernel uses ordered children:
kernel score < 0.95 ⇒ miss

takeaway

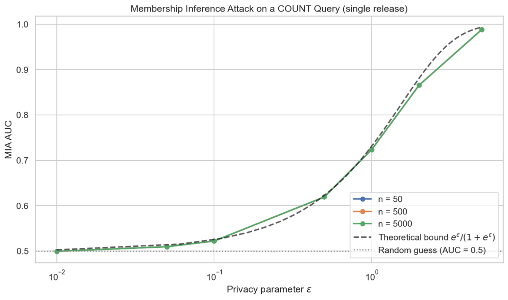
semantic similarity ≠ formal equivalence

- AI tools can measure how alike two queries look
- but DP requires provable equivalence of answers
- "AI-powered DP cache" alone is not DP-safe

Future direction: pair similarity tools with a symbolic equivalence prover (predicate normalization, range merging, alpha-conversion) before admitting any cache match.

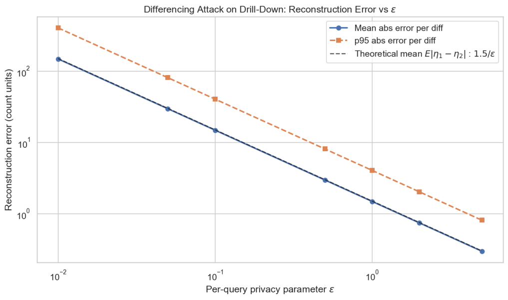
> 14 ::: privacy leakage, quantified

// single-query MIA AUC



empirical tracks theoretical $e^{\varepsilon}/(1+e^{\varepsilon})$ within 1%

// reconstruction error



differencing on 5-level drill-down, error $\sim \sqrt{2}/\varepsilon$

$\Rightarrow$ implementation matches the theory in the tight regimes.

> 15 ::: shadow MIA across W1–W4

logistic-regression classifier trained on simulated middleware outputs

AUC stays at 0.14–0.58 even though the cumulative-ε bound approaches 1.0

$\Rightarrow$ cumulative-ε is loose for realistic workloads.

> 16 ::: what we found – four takeaways

1. Workload-aware budget is structural, not a cache trick.
   $\mathbb{E}[\varepsilon_{\mathrm{wa}}] = \varepsilon_q \cdot \mathbb{E}[u_k]$ predicts within 3% on the main grid, exact on the mixed workload, same at SF=1 and SF=10.
2. The $1 - 1/k$ toy rule is the $\alpha \to \infty$ corner case.
   The model recovers it and the entire interior up to naive composition.
3. Cumulative-ε overestimates real attack success.
   Shadow MIA on W1–W4 stays at AUC 0.14–0.58 vs. a 1.0 cumulative bound – budget is over-provisioned in practice.
4. Semantic similarity $\neq$ equivalence.
   Tree-kernel + embedding admits false positives and misses textbook equivalences. "AI-powered DP" alone is not DP-safe.

> 17 ::: limitations + future work

// what this is not:

- Not a head-to-head reimplementation of PINQ / PrivateSQL / Chorus / DOP-SQL
- Not validated on a real workload trace (Zipf is a controlled sweep)
- Not a new cryptographic privacy mechanism

// concrete next steps:

- Cache re-release on budget surplus
- Rényi DP / zCDP composition
- Burstiness via renewal processes
- Real workload traces
- Joint budget-leakage bound
- Equivalence-checked semantic cache
- Multi-table / JOIN via residual sensitivity

> thank you _

questions ?

refik can öztaş // N25142279

repo: github.com/canoztas/cmp653-project